

PersistentVolume e PersistentVolumeClaim

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare il modello PV/PVC e la separazione di responsabilità tra amministratore e sviluppatore
 - Descrivere il ciclo di vita completo di un PersistentVolume e le politiche di reclaim
 - Distinguere i quattro access modes e scegliere quello corretto in base allo scenario
 - Configurare un PVC e comprendere il meccanismo di binding automatico verso un PV disponibile
 - Diagnosticare un PVC in stato **Pending** e identificarne la causa
 - Descrivere le best practice per la gestione del PVC su Azure ARO
-

Teoria e Concetti Chiave

Il Modello PV/PVC: Separazione dei Ruoli

Kubernetes separa chi gestisce lo storage fisico (l'amministratore) da chi lo consuma (lo sviluppatore/applicazione):

Ruolo	Risorsa	Responsabilità
Amministratore cluster	PersistentVolume (PV)	Crea/gestisce lo storage fisico (disco Azure, NFS, ecc.)
Sviluppatore/Applicazione	PersistentVolumeClaim (PVC)	Richiede storage con capacità e access mode desiderati
Kubernetes	Binding automatico	Abbina il PVC al PV compatibile

Analogia: Il PV è come un disco fisico acquistato dall'azienda IT. Il PVC è come un ticket di richiesta disco compilato dallo sviluppatore. Kubernetes è il responsabile IT che assegna il disco alla richiesta.

Ciclo di Vita di un PersistentVolume

Available → Bound → Released → (Retained | Deleted)

Stato	Significato
Available	PV disponibile, non ancora associato a nessun PVC
Bound	PV associato a un PVC specifico
Released	Il PVC è stato cancellato ma il PV non è ancora disponibile per nuovi PVC
Failed	Errore durante il reclaim automatico

persistentVolumeReclaimPolicy — cosa succede al PV quando il PVC viene cancellato:

Policy	Comportamento	Quando usare
Delete	Il PV e il disco sottostante vengono cancellati automaticamente	Storage dinamico (default per StorageClass)
Retain	Il PV rimane in stato Released , i dati sono preservati	Dati critici che richiedono verifica manuale prima della cancellazione
Recycle	Deprecato — pulizia superficiale del volume	Non usare

⚠ Attenzione: Con **Retain**, dopo la cancellazione del PVC il PV rimane in stato **Released** e **non è riutilizzabile automaticamente** da nuovi PVC. Per riutilizzarlo, l'amministratore deve rimuovere manualmente il **claimRef** dal PV.

Access Modes

Gli access modes definiscono come il volume può essere montato rispetto ai nodi del cluster:

Access Mode	Abbreviazione	Descrizione	Esempio storage
<code>ReadWriteOnce</code>	RWO	Un solo nodo in lettura/scrittura contemporaneamente	Azure Disk, iSCSI, local
<code>ReadOnlyMany</code>	ROX	Molti nodi in sola lettura simultaneamente	NFS (read-only), Azure Files
<code>ReadWriteMany</code>	RWX	Molti nodi in lettura/scrittura simultaneamente	Azure Files, NFS, CephFS
<code>ReadWriteOncePod</code>	RWOP	Un solo Pod in lettura/scrittura (OCP 4.13+)	Azure Disk CSI (garanzia più forte di RWO)

💡 **Suggerimento:** `ReadWriteOnce` significa "un nodo", non "un Pod". Se due Pod sono **sullo stesso nodo**, entrambi possono montare un volume RWO contemporaneamente. Per garantire accesso esclusivo a livello di Pod, usare `ReadWriteOncePod`.

Matching PVC → PV

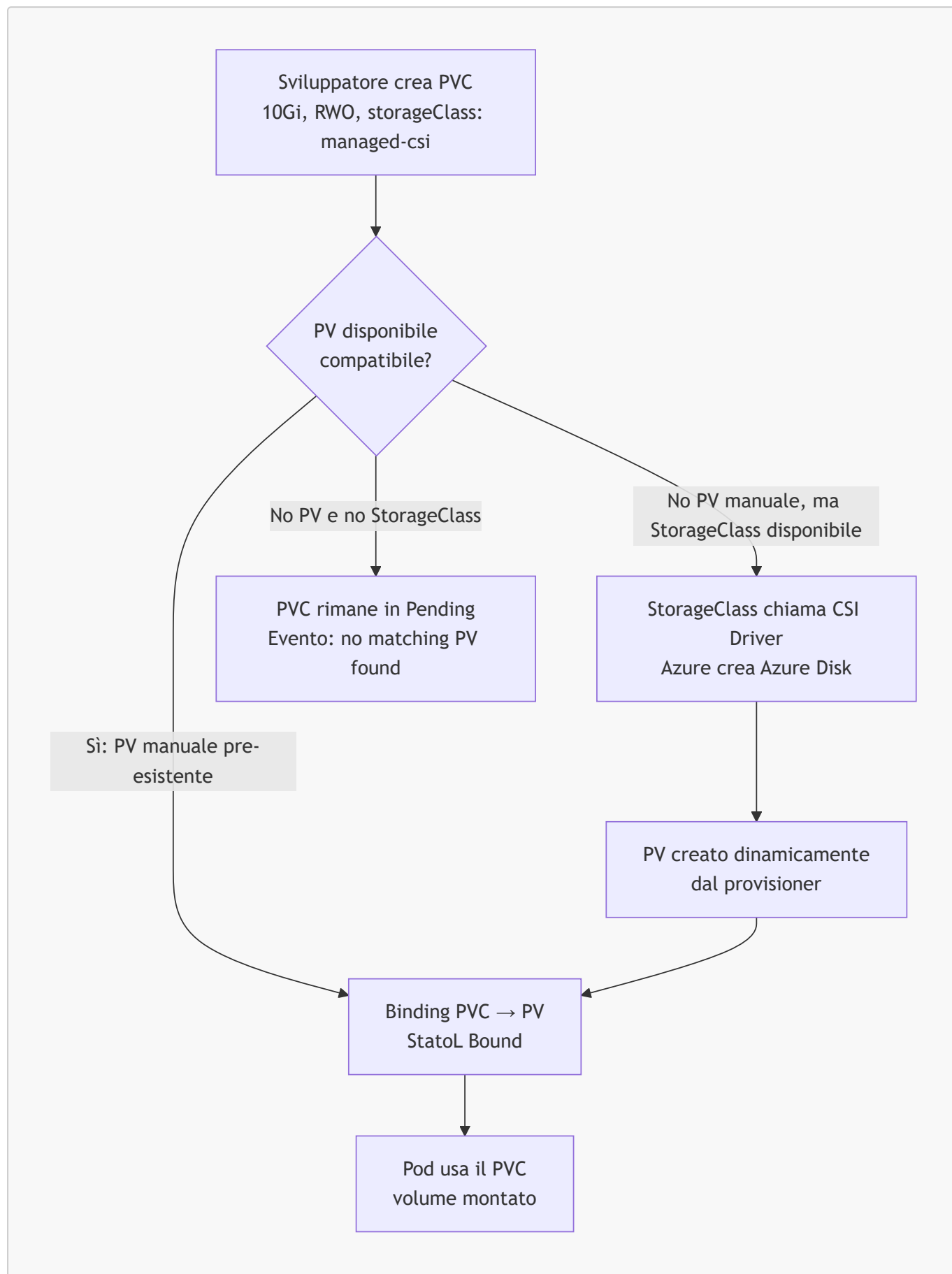
Quando viene creato un PVC, Kubernetes cerca un PV compatibile seguendo questi criteri nell'ordine:

1. **storageClassName:** deve corrispondere (o entrambi devono essere vuoti/default)
2. **Access mode:** il PV deve supportare almeno gli access modes richiesti dal PVC
3. **Capacità:** la capacità del PV deve essere $\geq$ quella richiesta dal PVC
4. **selector** (opzionale): label selector sul PVC deve corrispondere alle label del PV

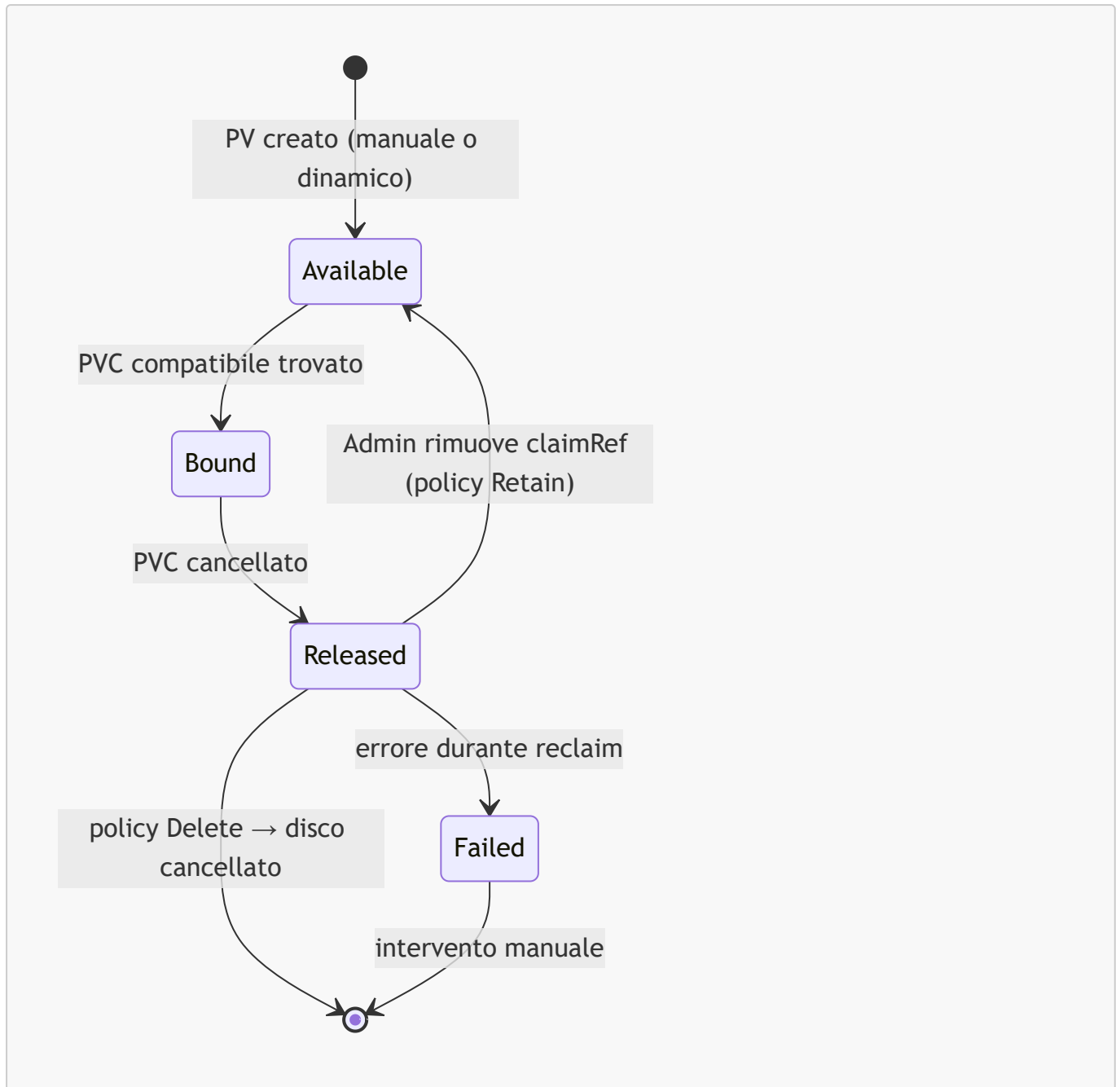
Se nessun PV compatibile è disponibile e non c'è una StorageClass configurata per il provisioning dinamico, il PVC rimane in stato `Pending`.

Diagrammi

Flowchart Binding PVC → PV



Ciclo di Vita di un PV (stateDiagram)



Comandi Pratici con Output Atteso

```
# Visualizzare tutti i PersistentVolume nel cluster
oc get pv
```

```
# Output atteso:
```

# NAME	CAPACITY	ACCESS MODES	RECLAIM
POLICY STATUS CLAIM	STORAGECLASS	AGE	
# pvc-1a2b3c4d-5e6f-7890-abcd-ef1234567890	10Gi	RWO	Delete
Bound myproject/webapp-data	managed-csi	2d	
# pvc-9f8e7d6c-5b4a-3210-fedc-ba9876543210	50Gi	RWO	Retain
Bound database-ns/mysql-data	managed-premium	5d	

```
# Visualizzare tutti i PVC in un namespace
oc get pvc -n myproject
```

```
# Output atteso:
```

# NAME	STATUS	VOLUME	CAPACITY
ACCESS MODES	STORAGECLASS	AGE	
# webapp-data	Bound	pvc-1a2b3c4d-5e6f-7890-abcd-ef1234567890	10Gi RWO
managed-csi	2d		
# shared-data	Bound	pvc-aa11bb22-cc33-dd44-ee55-ff6677889900	100Gi RWX
azurefile-csi	1d		

```
# Descrivere un PVC (utile per diagnosticare Pending)
oc describe pvc webapp-data -n myproject
```

```
# Output atteso (estratto):
```

```
# Name: webapp-data
# Namespace: myproject
# StorageClass: managed-csi
# Status: Bound
# Volume: pvc-1a2b3c4d-5e6f-7890-abcd-ef1234567890
# Capacity: 10Gi
# Access Modes: RWO
# Events:
#   Normal ProvisioningSucceeded 2d disk.csi.azure.com_csi-azuredisk-
controller-...
#   Successfully provisioned volume pvc-1a2b3c4d...
```

```
# Diagnosticare un PVC in Pending
oc describe pvc myapp-pvc -n myproject | grep -A 10 "Events:"
```

```
# Output atteso (PVC in Pending per capacità insufficiente):
# Events:
```

```
# Warning ProvisioningFailed 5m disk.csi.azure.com
# failed to provision volume: StorageClass "managed-csi" does not allow volume
expansion
```

```
# Visualizzare il dettaglio di un PV specifico
oc get pv pvc-1a2b3c4d-5e6f-7890-abcd-ef1234567890 -o yaml | grep -E
"capacity|accessModes|reclaimPolicy|status"

# Output atteso:
# capacity:
#   storage: 10Gi
# accessModes:
#   - ReadWriteOnce
# persistentVolumeReclaimPolicy: Delete
# status:
#   phase: Bound
```

```
# Trovare quale Pod usa un PVC
oc get pods -n myproject -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}
{range .spec.volumes[*]}{.persistentVolumeClaim.claimName}{"\n"}{end}{end}'

# Output atteso:
# webapp-7d8f9b-xkz2m    webapp-data
# mysql-0               mysql-data
```

```
# Cancellare un PVC (il PV viene cancellato se reclaimPolicy: Delete)
oc delete pvc webapp-data -n myproject

# Output atteso:
# persistentvolumeclaim "webapp-data" deleted
```

```
# Verificare lo stato dei PV dopo la cancellazione del PVC (policy Retain)
oc get pv | grep Released

# Output atteso:
# pvc-9f8e7d6c-...    50Gi    RW0    Retain    Released    database-ns/mysql-data
managed-premium    5d
```

Esempi YAML Completi

PV Manuale (tipo Azure Disk) + PVC + Pod

```
# PersistentVolume creato manualmente dall'amministratore
# NOTA: Su ARO usare il provisioning dinamico (StorageClass). Il PV manuale
# è mostrato qui a scopo didattico.
apiVersion: v1
kind: PersistentVolume
metadata:
  name: manual-azure-disk-pv
  labels:
    environment: production # ← label per selector nel PVC
    tier: database
spec:
  capacity:
    storage: 20Gi # ← capacità del volume
  accessModes:
    - ReadWriteOnce # ← un solo nodo in R/W
  persistentVolumeReclaimPolicy: Retain # ← dati preservati al delete del PVC
  storageClassName: managed-premium # ← deve corrispondere al PVC
  csi:
    driver: disk.csi.azure.com # ← CSI driver Azure Disk
    volumeHandle: /subscriptions/00000000-0000-0000-0000-
000000000000/resourceGroups/myRG/providers/Microsoft.Compute/disks/my-disk
    volumeAttributes:
      fsType: ext4
---
```

```
# PersistentVolumeClaim creato dallo sviluppatore
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-data
  namespace: database-ns
spec:
  storageClassName: managed-premium # ← deve corrispondere al PV
  accessModes:
    - ReadWriteOnce # ← deve essere supportato dal PV
  resources:
    requests:
      storage: 20Gi # ← il PV deve avere capacità ≥ 20Gi
  selector:
    matchLabels:
      environment: production # ← seleziona solo PV con questa label
      tier: database
---
```

```
# Pod che consuma il PVC
apiVersion: v1
kind: Pod
metadata:
  name: mysql-pod
```

```
namespace: database-ns
spec:
  containers:
  - name: mysql
    image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
    command: ["/bin/sh", "-c", "sleep 3600"]
    resources:
      requests:
        memory: "512Mi"
        cpu: "250m"
      limits:
        memory: "1Gi"
        cpu: "1000m"
    volumeMounts:
    - name: mysql-storage
      mountPath: /var/lib/mysql          # ← path nel container dove viene montato il
PVC
  volumes:
  - name: mysql-storage
    persistentVolumeClaim:
      claimName: mysql-data             # ← nome del PVC nel namespace
      readOnly: false                   # ← accesso in scrittura (default)
```

Note Specifiche per Azure ARO

Provisioning Dinamico su ARO

Su ARO, **non è necessario creare PV manualmente**. Il provisioning dinamico tramite StorageClass crea automaticamente il disco Azure quando viene creato un PVC:

```
# Verificare le StorageClass disponibili su ARO
oc get storageclass

# Output atteso:
# NAME                                PROVISIONER          RECLAIMPOLICY  VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION
# managed-csi (default)    disk.csi.azure.com   Delete
WaitForFirstConsumer      true
# managed-premium          disk.csi.azure.com   Delete
WaitForFirstConsumer      true
# azurefile-csi            file.csi.azure.com   Delete         Immediate
true
# azurefile-premium        file.csi.azure.com   Delete         Immediate
true
```

WaitForFirstConsumer su ARO

Su cluster ARO multi-zona, la StorageClass **managed-csi** usa **volumeBindingMode: WaitForFirstConsumer**. Questo significa che il PVC resta in **Pending** finché un Pod non viene schedulato su un nodo specifico, e solo allora il disco Azure viene creato nella zona corretta.

```
# PVC in Pending con WaitForFirstConsumer è normale finché non c'è un Pod
oc get pvc myapp-data -n myproject
# NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
# myapp-data Pending
# ← NORMALE: diventa Bound quando il Pod viene schedulato
```

Verificare il Disco Azure Creato Dinamicamente

```
# Trovare il nome del disco Azure associato a un PVC
oc get pv $(oc get pvc webapp-data -n myproject -o jsonpath='{.spec.volumeName}') \
-o jsonpath='{.spec.csi.volumeHandle}'

# Output atteso:
# /subscriptions/.../resourceGroups/aro-infra-
rg/providers/Microsoft.Compute/disks/pvc-1a2b3c4d-...
```

```
# Verificare il disco in Azure
az disk show \
  --ids "/subscriptions/.../resourceGroups/aro-infra-
rg/providers/Microsoft.Compute/disks/pvc-1a2b3c4d-..." \
  --query "{Name:name, Size:diskSizeGb, Sku:sku.name, State:diskState}" \
  --output table
```

 **Attenzione:** Non cancellare direttamente i dischi Azure dal portale Azure. Cancellare sempre i PVC da OpenShift: il CSI driver gestirà la cancellazione del disco sottostante (se `reclaimPolicy: Delete`).

Riepilogo dei Punti Chiave

- Il modello PV/PVC separa chi gestisce lo storage (amministratore → PV) da chi lo consuma (sviluppatore → PVC), favorendo la portabilità delle applicazioni
 - Il ciclo di vita di un PV è: **Available** → **Bound** → **Released** → (**Delete** | **Retain**) — la policy **Retain** preserva i dati ma richiede intervento manuale per riutilizzare il PV
 - **ReadWriteOnce** (RWO) significa un nodo (non un Pod); per accesso esclusivo a livello di Pod usare **ReadWriteOncePod** (RWOP, OCP 4.13+)
 - **ReadWriteMany** (RWX) richiede storage condiviso come Azure Files o NFS; Azure Disk non supporta RWX
 - Su ARO, il provisioning è sempre dinamico tramite StorageClass: non creare PV manualmente
 - Un PVC in **Pending** su cluster multi-zona con **WaitForFirstConsumer** è normale finché non viene creato il Pod che lo usa
 - Per diagnosticare un PVC in **Pending**, usare `oc describe pvc` e controllare la sezione **Events**
-

Domande di Verifica

1. Quale **persistentVolumeReclaimPolicy** preserva i dati del volume dopo la cancellazione del PVC, richiedendo intervento manuale dell'amministratore?

- A) **Delete**
- B) **Recycle**
- C) **Retain** ✓
- D) **Preserve**

2. Due Pod sullo stesso nodo montano lo stesso PVC con **accessMode: ReadWriteOnce**. Cosa succede?

- A) Il secondo Pod fallisce con errore "volume already mounted"
- B) Entrambi i Pod montano il volume correttamente, poiché RWO vincola il nodo, non il Pod ✓
- C) Il secondo Pod viene schedulato automaticamente su un altro nodo
- D) Kubernetes impone che solo un Pod per volta possa usare un PVC RWO

3. Un PVC su ARO rimane in stato **Pending** per 5 minuti dopo la creazione. Quale causa è più probabile?

- A) Il CSI driver di Azure Disk non è installato sul cluster
- B) Non è ancora stato creato un Pod che usa il PVC (StorageClass usa **waitForFirstConsumer**) ✓
- C) Il namespace non ha permessi per creare PVC
- D) La StorageClass non esiste nel cluster

4. Quale access mode è necessario per montare lo stesso PVC su Pod in esecuzione su nodi diversi contemporaneamente in lettura/scrittura?

- A) **ReadWriteOnce**
- B) **ReadOnlyMany**
- C) **ReadWriteMany** ✓
- D) **ReadWriteOncePod**

5. Su ARO, qual è la pratica raccomandata per creare storage persistente per un'applicazione?

- A) Creare manualmente un PV con il disco Azure pre-provisionato
- B) Usare un PVC con una StorageClass appropriata (es. **managed-csi**) e lasciare che il provisioner crei il disco automaticamente ✓
- C) Montare un **hostPath** che punta a una directory nel nodo worker
- D) Creare il disco Azure dal portale e montarlo direttamente nel Pod tramite **csi.volumeHandle**